

Topic 10: Abstract Classes and Pure Virtual Functions

10.1 Pure Virtual Functions

- A pure virtual function has no definition in the base class: `virtual void func() = 0;`
- Forces every concrete derived class to provide its own implementation
- Represents a contract -- every Shape must have `area()`, but Shape itself cannot define it meaningfully

```
class Shape {
public:
    virtual double area() const = 0; // pure virtual
    virtual double perimeter() const = 0; // pure virtual
    virtual ~Shape() {}
};
```

10.2 Abstract Classes

- A class with at least one pure virtual function is an abstract class
- You cannot create objects of an abstract class -- it serves only as a base
- Abstract classes define an interface that all derived classes must honour
- A derived class that does not override all pure virtual functions is also abstract

```
// Shape* s = new Shape(); // ERROR -- cannot instantiate abstract class

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
    double perimeter() const override { return 2*(w+h); }
};
```

10.3 Interfaces in C++

- C++ has no interface keyword -- interfaces are simulated with pure abstract classes
- A pure abstract class has only pure virtual functions and no data members
- A class can implement multiple interfaces (mitigates diamond problem since interfaces hold no data)

```

class Printable {
public:
    virtual void print() const = 0;
    virtual ~Printable() {}
};

class Serializable {
public:
    virtual string toJSON() const = 0;
    virtual ~Serializable() {}
};

class Report : public Printable, public Serializable {
public:
    void print() const override { cout << "Report" << endl; }
    string toJSON() const override { return "{\"type\":\"report\"}"; }
};

```

10.4 Design Principle: Program to Abstractions

- Write functions that accept base class pointers/references -- not concrete types
 - Code is then open for extension (add new derived classes) without modification
 - This is the Open/Closed Principle -- one of the SOLID design principles
-